

Linear Models for Regression Tasks

Albert Y. Kim

Outline

1. Basic review of linear regression
2. Parameters and optimization via gradient descent

Linear Regression

Linear regression is a supervised learning approach that models a numeric outcome as a linear combination of predictors:

$$Y = w_0 + w_1X_1 + w_2X_2 + \dots + w_pX_p + \epsilon$$

Linear Regression

Linear regression is a supervised learning approach that models a numeric outcome as a linear combination of predictors:

$$Y = w_0 + w_1X_1 + w_2X_2 + \dots + w_pX_p + \epsilon$$

Using matrix notation:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \epsilon$$

- ▶ In stats, the vector of *weights*, $\mathbf{w}$, are called the *slopes* (with w_0 being the *intercept*)
- ▶ In ML, the intercept w_0 , is often called the *bias* (or the *y-axis offset*)

Parameters and Cost Functions

For linear regression (and many other methods) the data are used to obtain an optimal estimate $\hat{\mathbf{w}}$ of the unknown $\mathbf{w}$ e.g. the least-squares estimates. This is done by **optimizing a cost function**:

$$Cost = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Parameters and Cost Functions

For linear regression (and many other methods) the data are used to obtain an optimal estimate $\hat{\mathbf{w}}$ of the unknown $\mathbf{w}$ e.g. the least-squares estimates. This is done by **optimizing a cost function**:

$$Cost = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

This function, *squared error cost*, expresses how close the model's predictions are to their corresponding observed values. In matrix notation:

$$Cost = \frac{1}{n} (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})$$

Optimizing the Weights

- ▶ The optimal set $\hat{\mathbf{w}}$ of weights are those that *minimize* the cost function, so we can use calculus to perform this minimization

Optimizing the Weights

- ▶ The optimal set $\hat{\mathbf{w}}$ of weights are those that *minimize* the cost function, so we can use calculus to perform this minimization
- ▶ Before jumping in, let's first do some algebraic rearrangement:

$$\begin{aligned} \text{Cost} &= \frac{1}{n}(\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T(\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}) \\ &= \frac{1}{n}\mathbf{y}^T\mathbf{y} + \frac{1}{n}(-2\mathbf{y}^T\mathbf{X}\hat{\mathbf{w}} + (\mathbf{X}\hat{\mathbf{w}})^T\mathbf{X}\hat{\mathbf{w}}) \\ &= \frac{1}{n}\mathbf{y}^T\mathbf{y} + \frac{1}{n}(-2\mathbf{y}^T\mathbf{X}\hat{\mathbf{w}} + \hat{\mathbf{w}}^T\mathbf{X}^T\mathbf{X}\hat{\mathbf{w}}) \end{aligned}$$

Optimizing the Weights

- ▶ Optimizing via calculus:
 - ▶ the **gradient** is the vector of partial derivatives with respect to each unknown variable in a function.
 - ▶ For linear regression, these unknowns are the model's weights (coefficients)

Optimizing the Weights

- ▶ Optimizing via calculus:
 - ▶ the **gradient** is the vector of partial derivatives with respect to each unknown variable in a function.
 - ▶ For linear regression, these unknowns are the model's weights (coefficients)
- ▶ Optimizing via linear algebra:
 - ▶ While it might not be immediately obvious, we can solve for a closed-form expression that minimizes the squared error cost function (the least squares solution)

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Optimizing the Weights

- ▶ Optimizing via calculus:
 - ▶ the **gradient** is the vector of partial derivatives with respect to each unknown variable in a function.
 - ▶ For linear regression, these unknowns are the model's weights (coefficients)
- ▶ Optimizing via linear algebra:
 - ▶ While it might not be immediately obvious, we can solve for a closed-form expression that minimizes the squared error cost function (the least squares solution)

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- ▶ For educational purposes in today's lab, we will ignore this closed-form solution and explore an alternative method for estimating an optimal set of weights called **gradient descent**

Simple Example

Say our true unknown model is a simple special case of linear regression with no intercept and a single weight parameter w_1 :

$$\begin{aligned} Y &= w_0 + w_1 X_1 + \epsilon \\ &= 0 + w_1 X_1 + \epsilon \\ &= 0 + 2.5 X_1 + \epsilon \\ &= 2.5 X_1 + \epsilon \end{aligned}$$

Simple Example

Say our true unknown model is a simple special case of linear regression with no intercept and a single weight parameter w_1 :

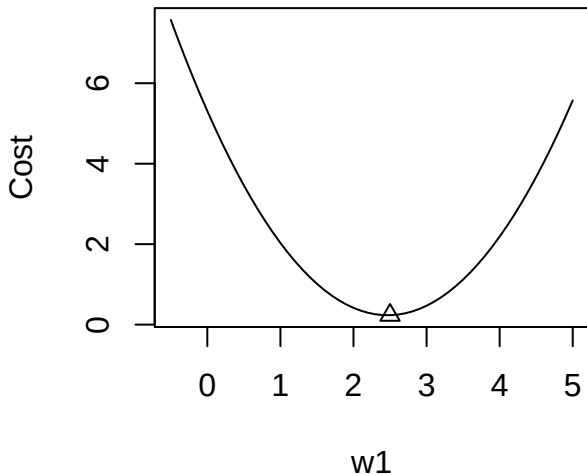
$$\begin{aligned}Y &= w_0 + w_1 X_1 + \epsilon \\&= 0 + w_1 X_1 + \epsilon \\&= 0 + 2.5 X_1 + \epsilon \\&= 2.5 X_1 + \epsilon\end{aligned}$$

We want to find the optimal estimate $\hat{w}_1$ of w_1 that minimizes the squared error cost function, which in this case simplifies to

$$Cost = \frac{1}{n}(\mathbf{y} - \mathbf{x}_1^T \hat{w}_1)^T (\mathbf{y} - \mathbf{x}_1^T \hat{w}_1)$$

Simple Example

We plot the cost as a function of w_1 below. Note that the optima is at the true unknown $w_1 = 2.5$, marked with a triangle.



Question to be solved for today

- ▶ Question: How do we find where this optima is located?

Question to be solved for today

- ▶ Question: How do we find where this optima is located?
- ▶ (One) Solution: Use the gradient descent algorithm!

Gradient Descent Algorithm

- ▶ The derivative is the slope of a function at a particular location, so we can use the gradient to gradually move towards the minimum of any (convex) cost function

Gradient Descent Algorithm

- ▶ The derivative is the slope of a function at a particular location, so we can use the gradient to gradually move towards the minimum of any (convex) cost function
- ▶ The **gradient descent algorithm** works to minimize the cost function using sequential updates. At step $j - 1$, we update our weights via

$$\hat{\mathbf{w}}^{(j)} = \hat{\mathbf{w}}^{(j-1)} - \alpha \frac{\partial \text{Cost}}{\partial \mathbf{w}}(\hat{\mathbf{w}}^{(j-1)})$$

Gradient Descent Algorithm

- ▶ The derivative is the slope of a function at a particular location, so we can use the gradient to gradually move towards the minimum of any (convex) cost function
- ▶ The **gradient descent algorithm** works to minimize the cost function using sequential updates. At step $j - 1$, we update our weights via

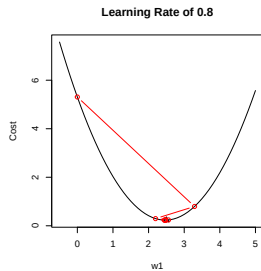
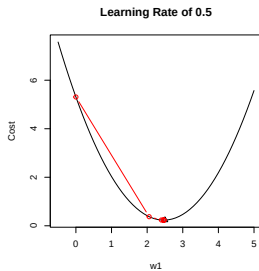
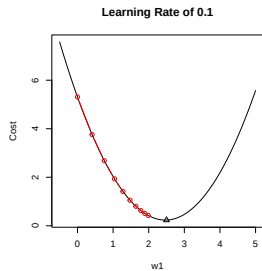
$$\hat{\mathbf{w}}^{(j)} = \hat{\mathbf{w}}^{(j-1)} - \alpha \frac{\partial \text{Cost}}{\partial \mathbf{w}}(\hat{\mathbf{w}}^{(j-1)})$$

- ▶ α is a tuning parameter that controls the *learning rate*, or how quickly to update the weight vector at each iteration
 - ▶ A small α requires many iterations for the algorithm to converge (reach the minimum)
 - ▶ A large α can overshoot the minimum, which can also cause convergence issues

Back to Simple Example: Learning Rates α

The graphs below illustrate gradient descent for our simple, one-parameter regression example with

- ▶ Starting at $w_1^{(0)} = 0$
- ▶ Three different learning rates: $\alpha = 0.1, 0.5, 0.8$
- ▶ 10 iterations each i.e. compute $w_1^{(1)}, w_1^{(2)}, \dots, w_1^{(10)}$



When does the algorithm stop?

Generally, gradient descent algorithms are programmed to end once the estimated parameters between step (j) and $(j - 1)$ change by no more than an acceptable tolerance (a small predetermined constant)

The Gradient for Linear Regression

- Recall that we can express the linear regression cost function as:

$$Cost = \frac{1}{n} \mathbf{y}^T \mathbf{y} + \frac{1}{n} (-2 \mathbf{y}^T \mathbf{X} \hat{\mathbf{w}} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})$$

The Gradient for Linear Regression

- ▶ Recall that we can express the linear regression cost function as:

$$Cost = \frac{1}{n} \mathbf{y}^T \mathbf{y} + \frac{1}{n} (-2 \mathbf{y}^T \mathbf{X} \hat{\mathbf{w}} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})$$

- ▶ Thus, the gradient is:

$$Gradient = \frac{-2}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X} \hat{\mathbf{w}})$$

The Gradient for Linear Regression

- ▶ Recall that we can express the linear regression cost function as:

$$Cost = \frac{1}{n} \mathbf{y}^T \mathbf{y} + \frac{1}{n} (-2 \mathbf{y}^T \mathbf{X} \hat{\mathbf{w}} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})$$

- ▶ Thus, the gradient is:

$$Gradient = \frac{-2}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X} \hat{\mathbf{w}})$$

- ▶ And our gradient descent updates look like:

$$\hat{\mathbf{w}}^{(j)} = \hat{\mathbf{w}}^{(j-1)} + \alpha \cdot \frac{2}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X} \hat{\mathbf{w}}^{(j-1)})$$

Algorithmic Run-Time

When using linear algebra as your model:

- ▶ Computing the gradient at each iteration required two vector-product calculations: $\mathbf{y}^T \mathbf{x}_1$ and $\mathbf{x}_1^T \mathbf{x}_1$
- ▶ Fortunately, these can both be computed ahead of time (rather than at each iteration) which makes algorithm very computationally efficient

Algorithmic Run-Time

When using linear algebra as your model:

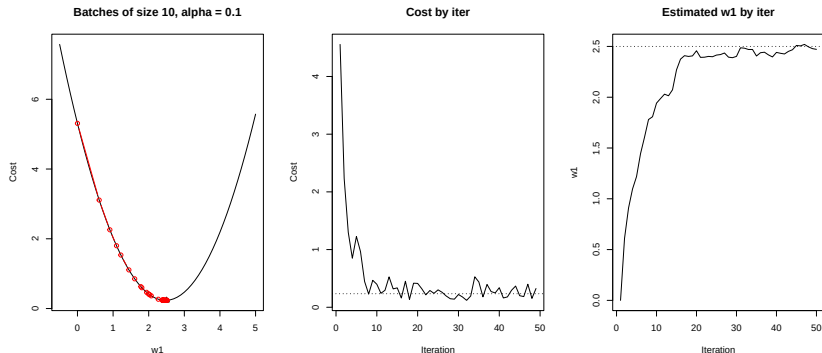
- ▶ Computing the gradient at each iteration required two vector-product calculations: $\mathbf{y}^T \mathbf{x}_1$ and $\mathbf{x}_1^T \mathbf{x}_1$
- ▶ Fortunately, these can both be computed ahead of time (rather than at each iteration) which makes algorithm very computationally efficient

When using other more complicated models:

- ▶ the parameter vector is involved in vector-product calculations within the gradient, so these vector-product calculations must be redone at each iteration
- ▶ In big-data settings, this computational challenge (among others) has led to the popularity of **stochastic gradient descent**

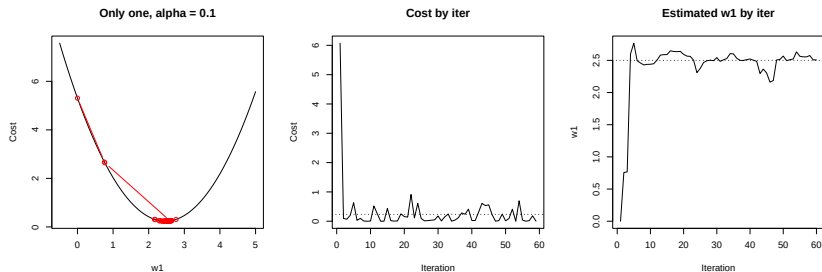
SKIP: Stochastic Gradient Descent

Stochastic Gradient Descent uses the same framework as gradient descent (updating parameters using the gradient to improve the cost function) but it does so using a *subset of training data* (or even just one data-point) at each iteration:



SKIP: Stochastic Gradient Descent

Even using *only one* data-point each iteration, stochastic gradient descent converges to the optimal value of w_1 (at least on average):



- ▶ This can be a huge computational benefit in big-data settings for models with complex gradients
- ▶ The algorithm's “noisy” behavior can help avoid local minima

Conclusion

This presentation briefly introduced *linear regression*, a modeling framework I'm assuming you're already familiar with:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \epsilon$$

Compared with other models we've discussed:

- ▶ Linear regression involves a structured set of parameters that must be learned from the data
- ▶ Gradient descent (or stochastic gradient descent) *can* be used to learn these parameters. - In practice, such as in `lm()` function in R, the linear algebra approach shown earlier is used.